

Windows Server Setup Guide

Last updated by | Haroun Ahmady | Apr 14, 2026 at 9:05 AM PDT

Windows Server Setup Guide

Overview

This guide covers the installation and configuration of the ClientIQ application on Windows Server. The application runs as a Node.js service behind Nginx reverse proxy.

Prerequisites

- Windows Server 2019 or 2022 (Desktop Experience or Core)
- Administrator access
- Network connectivity to SQL Server
- Internet access for package installation (or offline installers)

Step 1: Install Node.js

Option A: Online Installation

1. Download Node.js LTS (v20.x) from <https://nodejs.org/>
2. Run the installer with default options
3. Verify installation:

```
# Open PowerShell as Administrator
node --version
# Expected: v20.x.x

npm --version
# Expected: 10.x.x
```

Option B: Offline Installation

1. Copy the Node.js MSI installer to the server
2. Run silent installation:

```
msiexec /i node-v20.x.x-x64.msi /quiet /norestart
```

3. Add Node.js to system PATH if not added automatically:

```
$env:Path += ";C:\Program Files\nodejs"
[System.Environment]::SetEnvironmentVariable("Path", $env:Path, "Machine")
```

Step 2: Create Application Directory

```

# Create directory structure
New-Item -ItemType Directory -Path "C:\ClientIQ" -Force
New-Item -ItemType Directory -Path "C:\ClientIQ\app" -Force
New-Item -ItemType Directory -Path "C:\ClientIQ\logs" -Force
New-Item -ItemType Directory -Path "C:\ClientIQ\config" -Force
New-Item -ItemType Directory -Path "C:\ClientIQ\certs" -Force

# Set permissions (replace DOMAIN\ServiceAccount with your service account)
$acl = Get-Acl "C:\ClientIQ"
$rule = New-Object System.Security.AccessControl.FileSystemAccessRule(
    "NETWORK SERVICE", "FullControl", "ContainerInherit,ObjectInherit", "None", "Allow"
)
$acl.SetAccessRule($rule)
Set-Acl "C:\ClientIQ" $acl

```

Step 3: Deploy Application Files

Extract Application Package

```

# Copy deployment package to server
# Extract to application directory
Expand-Archive -Path "C:\Temp\clientiq-v2.0.0.zip" -DestinationPath "C:\ClientIQ\app" -Force

# Navigate to application directory
cd C:\ClientIQ\app

# Install production dependencies
npm ci --production

```

Verify File Structure

```

C:\ClientIQ\
├── app\
│   ├── dist\
│   │   ├── client\           # Frontend assets
│   │   └── server\          # Backend compiled code
│   ├── node_modules\        # Dependencies
│   ├── package.json
│   └── package-lock.json
├── logs\                    # Application logs
├── config\                  # Configuration files
└── certs\                   # SAML certificates

```

Step 4: Create Environment Configuration

Create the environment file at `C:\ClientIQ\config\.env` :

```

# Create environment file
@"
# =====
# ClientIQ Production Configuration
# =====

# Application
NODE_ENV=production
PORT=5000

# Database (SQL Server)
DATABASE_DIALECT=sqlserver
MSSQL_SERVER=sql-server.yourbank.local
MSSQL_DATABASE=ClientIQ
MSSQL_USER=svc_clientiq
MSSQL_PASSWORD=YourSecurePassword
MSSQL_PORT=1433
MSSQL_ENCRYPT=true
MSSQL_TRUST_SERVER_CERTIFICATE=false

# Session
SESSION_SECRET=your-64-character-random-string-here-use-crypto-to-generate

# SAML (See Document 04 for detailed configuration)
SAML_ENTRYPOINT=https://idp.yourbank.com/saml/sso
SAML_ISSUER=ClientIQ-Production
SAML_CALLBACK_URL=https://clientiq.yourbank.com/api/auth/saml/acs
SAML_LOGOUT_URL=https://idp.yourbank.com/saml/logout
SAML_CERT=MIIDxTCCA...
"@ | Out-File -FilePath "C:\ClientIQ\config\.env" -Encoding UTF8

```

Security Note: Protect the .env file:

```

# Remove inheritance and set explicit permissions
$acl = Get-Acl "C:\ClientIQ\config\.env"
$acl.SetAccessRuleProtection($true, $false)
$rule = New-Object System.Security.AccessControl.FileSystemAccessRule(
    "NETWORK SERVICE", "Read", "None", "None", "Allow"
)
$acl.SetAccessRule($rule)
$rule = New-Object System.Security.AccessControl.FileSystemAccessRule(
    "Administrators", "FullControl", "None", "None", "Allow"
)
$acl.SetAccessRule($rule)
Set-Acl "C:\ClientIQ\config\.env" $acl

```

Step 5: Install Windows Service

Using NSSM (Non-Sucking Service Manager)

1. Download NSSM from <https://nssm.cc/download>
2. Extract to C:\Tools\nssm

```

# Add NSSM to PATH
$env:Path += ";C:\Tools\nssm"
[System.Environment]::SetEnvironmentVariable("Path", $env:Path, "Machine")

# Install ClientIQ as a service
nssm install ClientIQ "C:\Program Files\nodejs\node.exe"
nssm set ClientIQ AppParameters "dist/server/index.js"
nssm set ClientIQ AppDirectory "C:\ClientIQ\app"
nssm set ClientIQ AppEnvironmentExtra "NODE_ENV=production"
nssm set ClientIQ AppEnvironmentExtra "+DOTENV_CONFIG_PATH=C:\ClientIQ\config\.env"

# Configure logging
nssm set ClientIQ AppStdout "C:\ClientIQ\logs\stdout.log"
nssm set ClientIQ AppStderr "C:\ClientIQ\logs\stderr.log"
nssm set ClientIQ AppStdoutCreationDisposition 4
nssm set ClientIQ AppStderrCreationDisposition 4
nssm set ClientIQ AppRotateFiles 1
nssm set ClientIQ AppRotateOnline 1
nssm set ClientIQ AppRotateBytes 10485760

# Configure service recovery
nssm set ClientIQ AppRestartDelay 5000
nssm set ClientIQ AppExit Default Restart

# Set service account
nssm set ClientIQ ObjectName "NETWORK SERVICE"

# Set display name and description
nssm set ClientIQ DisplayName "ClientIQ Banking Application"
nssm set ClientIQ Description "ClientIQ Client 360 Banking Platform"

# Start the service
nssm start ClientIQ

```

Alternative: Using node-windows

```

# Install node-windows globally
npm install -g node-windows

# Create service installation script
cd C:\ClientIQ\app

```

Create install-service.js :

```

const Service = require('node-windows').Service;

const svc = new Service({
  name: 'ClientIQ',
  description: 'ClientIQ Client 360 Banking Platform',
  script: 'C:\\ClientIQ\\app\\dist\\server\\index.js',
  nodeOptions: [],
  env: [
    { name: 'NODE_ENV', value: 'production' },
    { name: 'DOTENV_CONFIG_PATH', value: 'C:\\ClientIQ\\config\\.env' }
  ]
});

svc.on('install', () => {
  svc.start();
  console.log('Service installed and started');
});

svc.install();

```

Run installation:

```
node install-service.js
```

Step 6: Configure Firewall

```

# Allow Node.js application (internal only - Nginx will handle external)
New-NetFirewallRule -DisplayName "ClientIQ App" `
  -Direction Inbound `
  -LocalPort 5000 `
  -Protocol TCP `
  -Action Allow `
  -Profile Domain `
  -LocalAddress 127.0.0.1

# Allow outbound to SQL Server
New-NetFirewallRule -DisplayName "SQL Server Outbound" `
  -Direction Outbound `
  -RemotePort 1433 `
  -Protocol TCP `
  -Action Allow

```

Step 7: Configure Log Rotation

Create a scheduled task for log management:

```

# Create log cleanup script
@"
# Cleanup logs older than 30 days
Get-ChildItem -Path "C:\ClientIQ\logs" -Recurse -File |
    Where-Object { `$_ .LastWriteTime -lt (Get-Date).AddDays(-30) } |
    Remove-Item -Force

# Compress logs older than 7 days
Get-ChildItem -Path "C:\ClientIQ\logs" -Filter "*.log" |
    Where-Object { `$_ .LastWriteTime -lt (Get-Date).AddDays(-7) -and `$_ .Extension -ne ".zip" } |
    ForEach-Object {
        Compress-Archive -Path `$_ .FullName -DestinationPath "`$(`$_ .FullName).zip" -Force
        Remove-Item `$_ .FullName
    }
"@ | Out-File -FilePath "C:\ClientIQ\scripts\cleanup-logs.ps1" -Encoding UTF8

# Create scheduled task
$action = New-ScheduledTaskAction -Execute "PowerShell.exe" `
    -Argument "-NoProfile -ExecutionPolicy Bypass -File C:\ClientIQ\scripts\cleanup-logs.ps1"
$trigger = New-ScheduledTaskTrigger -Daily -At "3:00AM"
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -RunLevel Highest
$settings = New-ScheduledTaskSettingsSet -StartWhenAvailable

Register-ScheduledTask -TaskName "ClientIQ-LogCleanup" `
    -Action $action -Trigger $trigger -Principal $principal -Settings $settings

```

Step 8: Verify Installation

Check Service Status

```

# Check service status
Get-Service -Name "ClientIQ"

# View recent logs
Get-Content "C:\ClientIQ\logs\stdout.log" -Tail 50

```

Test Application Health

```

# Test internal health endpoint
Invoke-RestMethod -Uri "http://localhost:5000/api/health" -Method GET

# Expected response:
# { "status": "ok", "timestamp": "...", "database": "connected" }

```

Verify Database Connectivity

```

# Check database connection in logs
Select-String -Path "C:\ClientIQ\logs\stdout.log" -Pattern "Database|Connected|Error" -Context 0,2

```

Service Management Commands

```

# Start service
Start-Service -Name "ClientIQ"
# or: nssm start ClientIQ

# Stop service
Stop-Service -Name "ClientIQ"
# or: nssm stop ClientIQ

# Restart service
Restart-Service -Name "ClientIQ"
# or: nssm restart ClientIQ

# View service status
Get-Service -Name "ClientIQ" | Format-List *

# View recent application logs
Get-Content "C:\ClientIQ\logs\stdout.log" -Tail 100 -Wait

```

Troubleshooting

Service Won't Start

1. Check Windows Event Viewer: `eventvwr.msc` > Windows Logs > Application
2. Check application logs: `C:\ClientIQ\logs\stderr.log`
3. Verify environment file permissions
4. Test Node.js manually:

```

cd C:\ClientIQ\app
$env:DOTENV_CONFIG_PATH = "C:\ClientIQ\config\.env"
node dist/server/index.js

```

Database Connection Issues

1. Verify SQL Server network connectivity:

```
Test-NetConnection -ComputerName "sql-server.yourbank.local" -Port 1433
```

2. Check SQL Server authentication:

```

# Install SqlServer module if needed
Install-Module -Name SqlServer -Scope CurrentUser

# Test connection
Invoke-Sqlcmd -ServerInstance "sql-server.yourbank.local" `
  -Database "ClientIQ" `
  -Username "svc_clientiq" `
  -Password "YourPassword" `
  -Query "SELECT 1"

```

Memory Issues

Monitor memory usage:

```
Get-Process -Name "node" | Select-Object WorkingSet64, CPU
```

Configure Node.js memory limit if needed in the service configuration:

```
nssm set ClientIQ AppParameters "--max-old-space-size=4096 dist/server/index.js"
```

Next Steps

1. [Configure Nginx Reverse Proxy](#)
2. [Set Up SSL Certificate](#)
3. [Configure SAML Authentication](#)